

Theory of computation

Sajed kittanh :202411395

Supervisor :Dr.Amjad Ratroot

In this notebook, we will implement the following automata:

1. DFA
2. NFA
3. GNFA
4. PDA
5. Turing Machine

We will use **Unary Addition**.

Example:

$111 + 11 = 11111$

This means:

$3 + 2 = 5$

1. Deterministic finite Automita (DFA)

```
In [31]: def DFA(s):
    transitions = {
        ("q0", "1"): "q1",

        ("q1", "1"): "q1",
        ("q1", "+"): "q2",

        ("q2", "1"): "q3",

        ("q3", "1"): "q3",
        ("q3", "="): "q4",

        ("q4", "1"): "q5",

        ("q5", "1"): "q5",
    }

    state="q0"

    for c in s:
        state=transitions.get((state, c), "dead")

    return state=="q5"
```

example

```
In [32]: accepted = "111+11=11111"
         rejected = "111++11=11111"
```

```
In [33]: print("DFA result for accepted:", DFA(accepted))
         print("DFA result for rejected:", DFA(rejected))
```

DFA result for accepted: True

DFA result for rejected: False

A DFA is a finite automaton where each state has exactly one transition for each input symbol.

In this example, the DFA will check if the input has the correct unary addition format:

1+1=1

Example format:

111+11=11111

The DFA checks only the structure, not whether the addition result is correct.

2. Non Deterministic finite Automata (NFA)

```
In [34]: def NFA(s):
         nfa = { "q0":{"1":{"q1"}},
                 "q1":{"1": {"q1"}, "+":{"q2"}},
                 "q2":{"1": {"q3"}},
                 "q3":{"1":{"q3"}},
                 "=": {"q4"} },
                 "q4":{"1": {"q5"}},
                 "q5":{"1":{"q5"}}
                 }

         current_states = {"q0"}
         for c in s:
             next_states = set()
             for state in current_states:
                 next_states |= nfa.get(state, {}).get(c, set())
             current_states = next_states
         return "q5" in current_states
```

example

```
In [35]: accepted="11+111=11111"
         rejected="11+111"
```

```
In [36]: print(NFA(accepted))
         print(NFA(rejected))
```

True

False

The rejected example is missing the equal sign = and the result part.

The NFA reads the input symbol by symbol.

It accepts strings that have this format:

first unary number + second unary number = result unary number

Accepted example:

11+111=11111

Rejected example:

11+111

The rejected example is missing the equal sign = and the result part.

3.Generalized Non Deterministic finite automita

```
In [37]: import re
def GNFA(s):
    templete=r"1+\+1+=1+"
    return bool(re.fullmatch(templete,s))
```

example

```
In [38]: accepted = "111+1=1111"
rejected = "+111=111"
```

```
In [39]: print("GNFA result for accepted:", GNFA(accepted))
print("GNFA result for rejected:", GNFA(rejected))
```

GNFA result for accepted: True

GNFA result for rejected: False

The GNFA uses a regular expression to check the format of the input.

Accepted example:

111+1=1111

This is accepted because it follows the correct unary addition format.

Rejected example:

+111=111

This is rejected because there is no unary number before the plus sign.

4. Pushdown automita

```
In [40]: def PDA_accepts(s):
    stack = ["$"]

    state="first"
    seen_first=False
    seen_second=False
    seen_result=False

    for c in s:
```

```

if state == "first":
    if c == "1":
        stack.append("X")
        seen_first = True

    elif c=="+" and seen_first:
        state = "second"

    else:
        return False

elif state == "second":
    if c == "1":
        stack.append("X")
        seen_second = True

    elif c=="=" and seen_second:
        state="result"

    else:
        return False

elif state=="result":
    if c=="1":
        seen_result=True

        if len(stack)==1:
            return False

        stack.pop()

    else:
        return False

return state == "result" and seen_result and stack==["$"]

```

The PDA uses a stack to verify the addition.

For this input:

111+11=11111

The PDA pushes one symbol `X` for each `1` before the equal sign.

Before `=`, we have:

111+11

So the PDA pushes 5 symbols onto the stack.

Then, after the equal sign, the PDA reads:

11111

For each `1` in the result, the PDA pops one `X` from the stack.

If the stack becomes empty at the end, the addition is correct.

Accepted example:

111+11=11111

Rejected example:

111+11=1111

The rejected example is incorrect because:

3 + 2 = 5

but the result has only 4 ones.

```
In [41]: accepted = "111+11=11111"
         rejected = "111+11=1111"
```

```
In [42]: print("Accepted example:", accepted, "=>", PDA_accepts(accepted))
         print("Rejected example:", rejected, "=>", PDA_accepts(rejected))
```

Accepted example: 111+11=11111 => True

Rejected example: 111+11=1111 => False

The machine scans the tape until it finds the plus sign `+`.

Then it removes the plus sign and shifts the second number one position to the left.

So:

111+11

becomes:

11111

,

5. Turing Machine

```
In [43]: def Tm_unary_add(input):
         if not re.fullmatch(r"1+\+1+", input):
             raise ValueError("Rejected input. Use format like: 111+11")

         tape = list(input) + ["_"]
         head = 0
         state = "scan_plus"

         while state != "halt":

             if state == "scan_plus":
                 if tape[head] == "+":
                     state = "shift_left"
                 else:
                     head += 1

             elif state == "shift_left":
                 tape[head] = tape[head + 1]

                 if tape[head] == "_":
                     state = "halt"
                 else:
                     head += 1
```

```
return "".join(c for c in tape if c == "1")
```

example

```
In [44]: accepted = "111+11"
         rejected = "111++11"
```

```
In [45]: print("Accepted example:", accepted, "=>", Tm_unary_add(accepted))

         try:
             print("Rejected example:", rejected, "=>", Tm_unary_add(rejected))
         except ValueError as e:
             print("Rejected example:", rejected, "=>", e)
```

Accepted example: 111+11 => 11111

Rejected example: 111++11 => Rejected input. Use format like: 111+11

The Turing Machine receives the input:

111+11

The machine scans the tape until it finds the plus sign + .

Then it removes the plus sign and shifts the second number one position to the left.

So:

111+11

becomes:

11111

This is unary addition because combining the two groups of 1 s gives the sum.

Accepted example:

111+11

Output:

11111

Rejected example:

111++11

This is rejected because the input format is invalid.

This is unary addition because combining the two groups of 1 s gives the sum.

Accepted example:

111+11

Output:

11111

Rejected example:

111++11

This is rejected because the input format is invalid.

conclusion:

Machine	Purpose
DFA	Checks unary addition format
NFA	Checks unary addition format
GNFA	Checks unary addition format using regular expression
PDA	Verifies actual unary addition using stack
Turing Machine	Performs unary addition process

DFA, NFA, and GNFA are finite-state machines, so they cannot verify unlimited addition correctness.

The PDA can verify unary addition using a stack.

The Turing Machine can perform the addition operation directly.

The End

```
In [ ]: #convert to html
!jupyter nbconvert --to html Theory.ipynb
```

```
In [ ]:
```